
KHẢO SÁT TOÀN DIỆN SỰ PHÁT TRIỂN CỦA YOLO

Từ YOLOv1 (2015) đến YOLO26 (2026)

Kiến trúc – Phương pháp – Kết quả – Xu hướng

Nguyễn Cao Trị

Ngày biên soạn: Ngày 14 tháng 5 năm 2026

Mục lục

1	MỞ ĐẦU	4
1.1	Giới thiệu	4
1.2	Phạm vi báo cáo	4
2	NỀN TẢNG LÝ THUYẾT	5
2.1	Kiến trúc chung (Pipeline)	5
2.2	Khái niệm & Thước đo cốt lõi	5
3	Thế Hệ Đầu: YOLOv1 - YOLOv4	6
3.1	YOLOv1 (2015) — Khởi nguồn	6
3.1.1	Ý tưởng cốt lõi	6
3.1.2	Kiến trúc	6
3.1.3	Nhận xét	7
3.2	YOLOv2 / YOLO9000 (2016) — Better, Faster, Stronger	7
3.2.1	Ý tưởng cốt lõi	7
3.2.2	Kiến trúc	8
3.2.3	Nhận xét	8
3.3	YOLOv3 (2018) — Multi-Scale Detection	8
3.3.1	Ý tưởng cốt lõi	8
3.3.2	Kiến trúc	9
3.3.3	Nhận xét	9
3.4	YOLOv4 (2020) — Tập đại thành	9
3.4.1	Ý tưởng cốt lõi	10
3.4.2	Kiến trúc	10
3.4.3	Nhận xét	11
4	THẾ HỆ GIỮA: YOLOv5 – YOLOv8 (2020–2023)	12
4.1	YOLOv5 (2020) — Dân chủ hóa YOLO	12
4.1.1	Ý tưởng cốt lõi	12
4.1.2	Kiến trúc	12
4.1.3	Nhận xét	12
4.2	YOLOv6 (2022) — Industrial Deployment	13

4.2.1	Ý tưởng cốt lõi	13
4.2.2	Kiến trúc	13
4.2.3	Nhận xét	13
4.3	YOLOv7 (2022) — SOTA Accuracy	14
4.3.1	Ý tưởng cốt lõi	14
4.3.2	Kiến trúc	14
4.3.3	Nhận xét	14
4.4	YOLOv8 (2023) — Đa năng nhất	15
4.4.1	Ý tưởng cốt lõi	15
4.4.2	Kiến trúc	15
4.4.3	Nhận xét	15
5	THẾ HỆ MỚI: YOLOv9 – YOLOv12 (2024–2025)	16
5.1	YOLOv9 (2024) — Chống mất thông tin	16
5.1.1	Ý tưởng cốt lõi	16
5.1.2	Kiến trúc	16
5.1.3	Nhận xét	16
5.2	YOLOv10 (2024) — NMS-Free	17
5.2.1	Ý tưởng cốt lõi	17
5.2.2	Kiến trúc	17
5.2.3	Nhận xét	17
5.3	YOLO11 (2024) — Nâng cấp toàn diện	18
5.3.1	Ý tưởng cốt lõi	18
5.3.2	Kiến trúc	18
5.3.3	Nhận xét	18
5.4	YOLOv12 (2025) — Kỷ nguyên Attention	18
5.4.1	Ý tưởng cốt lõi	19
5.4.2	Kiến trúc	19
5.4.3	Nhận xét	19
6	HIỆN TẠI VÀ TƯƠNG LAI: YOLO26 (2026)	20
6.1	Ý tưởng cốt lõi	20
6.2	Kiến trúc	20
6.3	Nhận xét	20
6.4	MuSGD (Muon-enhanced SGD)	21
6.5	STAL (Small-Target-Aware Label Assignment)	22
6.6	ProgLoss (Progressive Loss Balancing)	22
6.7	Kết quả	23
6.8	Tổng kết YOLO26	23

7	BƯỚC NGOẶT OPEN-VOCABULARY	25
7.1	YOLO-World (2024) — Tiên phong	25
7.1.1	Ý tưởng cốt lõi	25
7.1.2	Kiến trúc	25
7.1.3	Nhận xét	25
7.2	YOLOE (2025) — Real-Time Seeing Anything	26
7.2.1	Ý tưởng cốt lõi	26
7.2.2	Kiến trúc	26
7.2.3	Nhận xét	26
8	CÁC BIẾN THỂ YOLO	27
8.1	YOLOX (2021) — Anchor-Free tiên phong	27
8.1.1	Ý tưởng cốt lõi	27
8.1.2	Kiến trúc	27
8.1.3	Nhận xét	27
8.2	PP-YOLOE (2022) — Baidu Industrial	28
8.2.1	Ý tưởng cốt lõi	28
8.2.2	Kiến trúc	28
8.2.3	Nhận xét	28
8.3	YOLO-NAS (2023) — AI thiết kế AI	28
8.3.1	Ý tưởng cốt lõi	28
8.3.2	Kiến trúc	29
8.3.3	Nhận xét	29
9	SƠ ĐỒ PHẢ HỆ YOLO VÀ KẾT LUẬN	30
9.1	Sơ đồ phả hệ YOLO (2015–2026)	30
9.2	Bảng tổng quan tất cả phiên bản	31
9.3	Xếp hạng mAP COCO	32
9.4	Tiến hóa công nghệ qua các phiên bản	33
9.5	Kết luận	33
	Phụ lục: Bảng thuật ngữ	36

CHƯƠNG 1: MỞ ĐẦU

1.1 Giới thiệu

Năm 2015, YOLO (You Only Look Once) tạo bước ngoặt lịch sử khi gom toàn bộ quá trình nhận diện vật thể thành một bước duy nhất (**one-stage**), giúp AI lần đầu tiên đạt tốc độ thời gian thực (real-time).

Qua 11 năm (2015–2026) với 14 phiên bản, YOLO đã "dân chủ hóa" thị giác máy tính, đưa công nghệ từ phòng thí nghiệm ứng dụng thẳng vào y tế, xe tự hành và các thiết bị Edge AI siêu nhỏ.

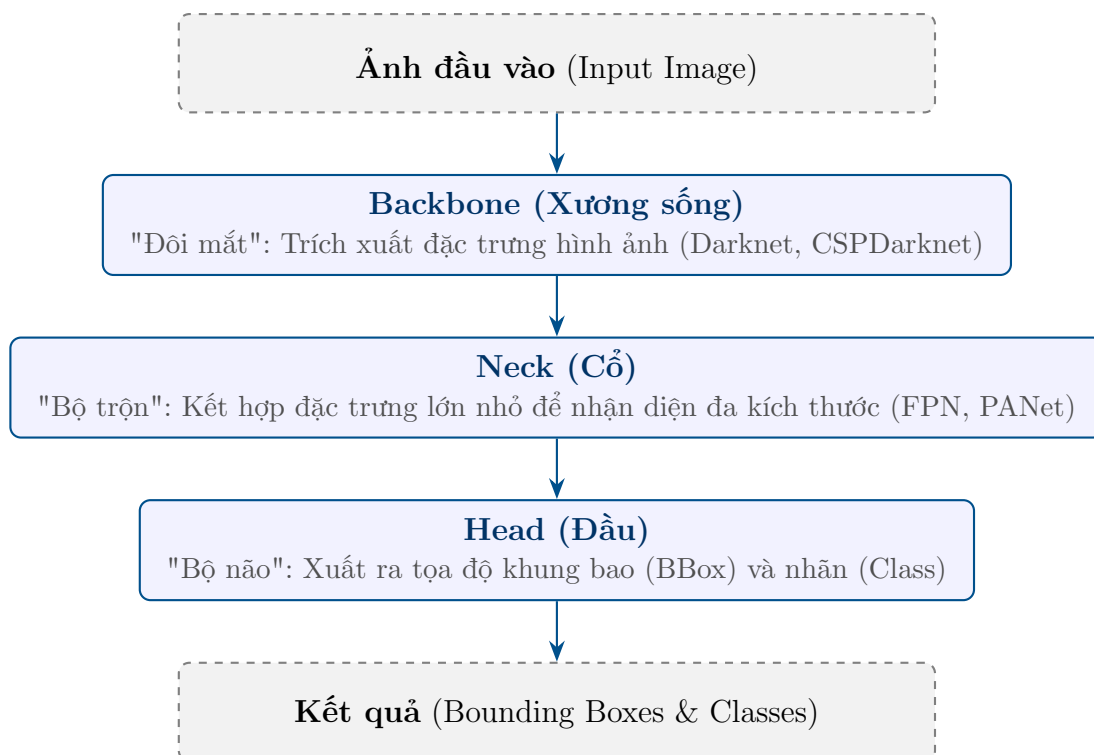
1.2 Phạm vi báo cáo

Báo cáo này là bức tranh toàn cảnh về sự tiến hóa của YOLO, tập trung vào ba trọng tâm:

- **Kiến trúc dòng chính:** Đánh giá đột phá kỹ thuật qua từng thời kỳ: khai mở (v1–v4), bùng nổ (v5–v8), nâng cấp Attention/NMS-free (v9–v12) và sự tinh giản Edge-First (YOLO26).
- **Biến thể nổi bật:** Khảo sát các nhánh rẽ công nghiệp có ảnh hưởng lớn (YOLOX, PP-YOLOE, YOLO-NAS).
- **So sánh đa chiều:** Phân tích sự đánh đổi (trade-off) giữa Độ chính xác (mAP) và Tốc độ (FPS/MACs) nhằm đưa ra định hướng lựa chọn mô hình thực tiễn.

CHƯƠNG 2: NỀN TẢNG LÝ THUYẾT

2.1 Kiến trúc chung (Pipeline)



Hình 2.1: Pipeline 3 thành phần tiêu chuẩn của mô hình YOLO

2.2 Khái niệm & Thước đo cốt lõi

- **mAP (mean Average Precision):** Thang đo độ chính xác. Tiêu chuẩn vàng hiện nay là **mAP@50:95** đo trên tập dữ liệu **MS COCO**.
- **Anchor Boxes (Khung neo):** Khung đoán mẫu. Các đời cũ (v2-v7) bị lệ thuộc vào khung này (Anchor-based). Từ v8 trở đi đã bỏ hẳn để linh hoạt hơn (Anchor-free).
- **NMS (Non-Maximum Suppression):** Bộ lọc xóa các dự đoán trùng lặp. Đã bị khai tử từ YOLOv10 (NMS-free) để tối đa hóa tốc độ.

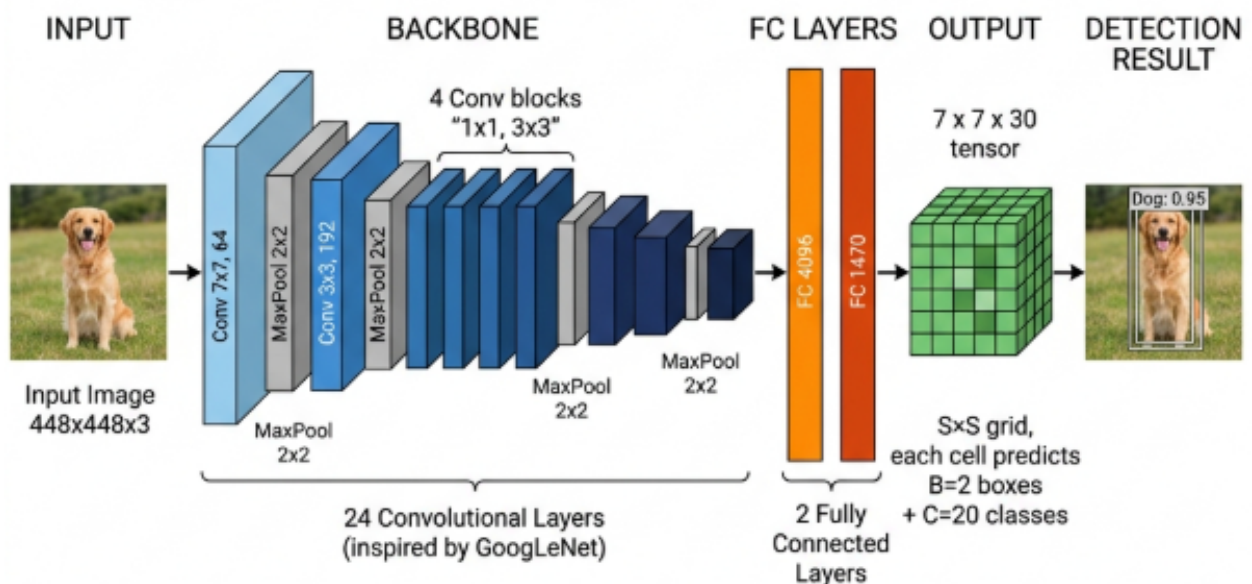
CHƯƠNG 3:

Thế Hệ Đầu: YOLOv1 - YOLOv4

3.1 YOLOv1 (2015) — Khởi nguồn

Paper: “You Only Look Once: Unified, Real-Time Object Detection” (CVPR 2016) [1]

Tác giả: Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi



Hình 3.1: Kiến trúc YOLOv1: 24 Conv layers + 2 FC layers. Input $448 \times 448 \rightarrow$ Output tensor $7 \times 7 \times 30$ (dựa theo Figure 3, Redmon et al. 2016 [1]).

3.1.1 Ý tưởng cốt lõi

Coi object detection là **bài toán hồi quy duy nhất**: từ pixel ảnh $\rightarrow$ bounding boxes + class probabilities, chỉ qua **một lần forward pass**. Chia ảnh thành lưới $S \times S$ ($S = 7$), mỗi ô dự đoán $B = 2$ boxes và $C = 20$ classes [1].

3.1.2 Kiến trúc

24 lớp convolutional + 2 lớp fully connected. Cảm hứng từ GoogLeNet. Output: tensor $7 \times 7 \times 30$. Fast YOLO: 9 conv layers.

3.1.3 Nhận xét

Có gì mới?

- Là mô hình **one-stage đầu tiên** trong lịch sử: gom detection thành một phép hồi quy duy nhất, bỏ qua hoàn toàn bước đề xuất vùng (region proposal) vốn rất chậm của R-CNN.
- Đạt **45 FPS** (phiên bản Fast YOLO: 155 FPS) — nhanh gấp **90×** so với Fast R-CNN (0.5 FPS), lần đầu tiên cho phép chạy detection trên video thời gian thực.
- Nhìn toàn bộ ảnh cùng lúc nên hiểu ngữ cảnh tốt hơn, giảm mạnh lỗi nhận nhầm nền thành vật thể (false positive background giảm hơn 50% so với R-CNN).

Hạn chế:

- Lưới 7×7 quá thô $\rightarrow$ mỗi ô chỉ dự đoán được 2 box và 1 class duy nhất. Khi nhiều vật nhỏ nằm cùng một ô (ví dụ: đàn chim đậu sát nhau), model chỉ phát hiện được 1 con và bỏ sót phần còn lại.
- Độ chính xác định vị (localization) kém, đặc biệt với vật thể có tỉ lệ khung hình bất thường (quá dài hoặc quá rộng).

Ví dụ: Hãy hình dung bạn cần tìm tất cả các chú chó trong một bức ảnh. Mô hình cũ (R-CNN) dùng kính lúp soi từng góc ảnh — chính xác nhưng mất cả phút. YOLOv1 thì kẻ ảnh thành bàn cờ 7×7 , lùi lại một bước nhìn tổng thể và đánh dấu ngay ô nào chứa chó — chỉ trong $\frac{1}{45}$ giây. Đổi lại, nếu hai chú chó đứng quá gần nhau trong cùng một ô, YOLOv1 chỉ “thấy” được một con.

3.2 YOLOv2 / YOLO9000 (2016) — Better, Faster, Stronger

Paper: “YOLO9000: Better, Faster, Stronger” (CVPR 2017) [2]

Tác giả: Joseph Redmon, Ali Farhadi

3.2.1 Ý tưởng cốt lõi

Và triệt để các lỗi của v1 bằng 7 kỹ thuật cải tiến tích lũy, đồng thời mở rộng khả năng nhận diện lên hơn 9000 loại vật thể thông qua cơ chế WordTree.

3.2.2 Kiến trúc

Backbone mới **Darknet-19** (19 conv + 5 maxpool), chỉ tốn 5.58B FLOPs (VGG-16: 30.69B). Lần đầu giới thiệu **Anchor Boxes** — 5 khung neo được tìm tự động bằng k-means trên dataset.

3.2.3 Nhận xét

Có gì mới?

- Thêm **Batch Normalization** vào mọi conv layer → bỏ Dropout, tăng +2% mAP.
- Giới thiệu **Anchor Boxes**: thay vì đoán tọa độ box từ con số 0 (như v1), model chỉ cần “tinh chỉnh” các khung mẫu có sẵn → recall tăng +7%.
- Huấn luyện trên **nhiều kích thước ảnh** (multi-scale: 320–608) → model linh hoạt hơn.
- Tổng cộng: mAP nhảy từ 63.4% lên **76.8%** (+13.4%) trên VOC 2007.

Hạn chế:

- Anchor boxes cố định theo dataset → chuyển sang dataset mới phải chạy lại k-means.
- Vẫn chỉ dự đoán ở **1 scale duy nhất** (13×13), nên vật nhỏ vẫn bị bỏ sót.

Ví dụ: Nếu YOLOv1 giống “vẽ tay tự do” khung bao quanh vật (dễ méo, dễ sai kích thước), thì YOLOv2 giống “chọn từ 5 khuôn mẫu có sẵn” rồi chỉnh nhẹ — nhanh hơn và chuẩn hơn rất nhiều.

3.3 YOLOv3 (2018) — Multi-Scale Detection

Paper: “YOLOv3: An Incremental Improvement” (Tech Report) [3]

Tác giả: Joseph Redmon, Ali Farhadi (*paper cuối cùng của Redmon trước khi dừng nghiên cứu CV vì lo ngại đạo đức*)

3.3.1 Ý tưởng cốt lõi

Dự đoán vật thể ở **3 kích thước khác nhau** (3 scales) theo phong cách Feature Pyramid Network (FPN), vá dứt điểm lỗi “mù vật nhỏ” của v1 và v2.

3.3.2 Kiến trúc

Backbone mới **Darknet-53** (53 conv layers + residual connections). Chính xác ngang ResNet-152 nhưng **nhANH GẤP 2×** (78 vs 37 FPS). Head dự đoán ở 3 feature maps: 13×13 , 26×26 , 52×52 với 3 anchors/scale = 9 anchors tổng.

3.3.3 Nhận xét

Có gì mới?

- **3 scales** detection: feature map 52×52 chuyên bắt vật nhỏ, 13×13 chuyên bắt vật lớn $\rightarrow$ bao phủ mọi kích thước.
- Thay **Softmax** bằng **Sigmoid** cho classification $\rightarrow$ hỗ trợ multi-label (một vật có thể vừa là “người” vừa là “phụ nữ”).
- Residual connections giúp train mạng sâu 53 layers mà không bị vanishing gradient.
- mAP@50 đạt **57.9%** trên COCO, ngang RetinaNet nhưng nhanh gấp **3.8×**.

Hạn chế:

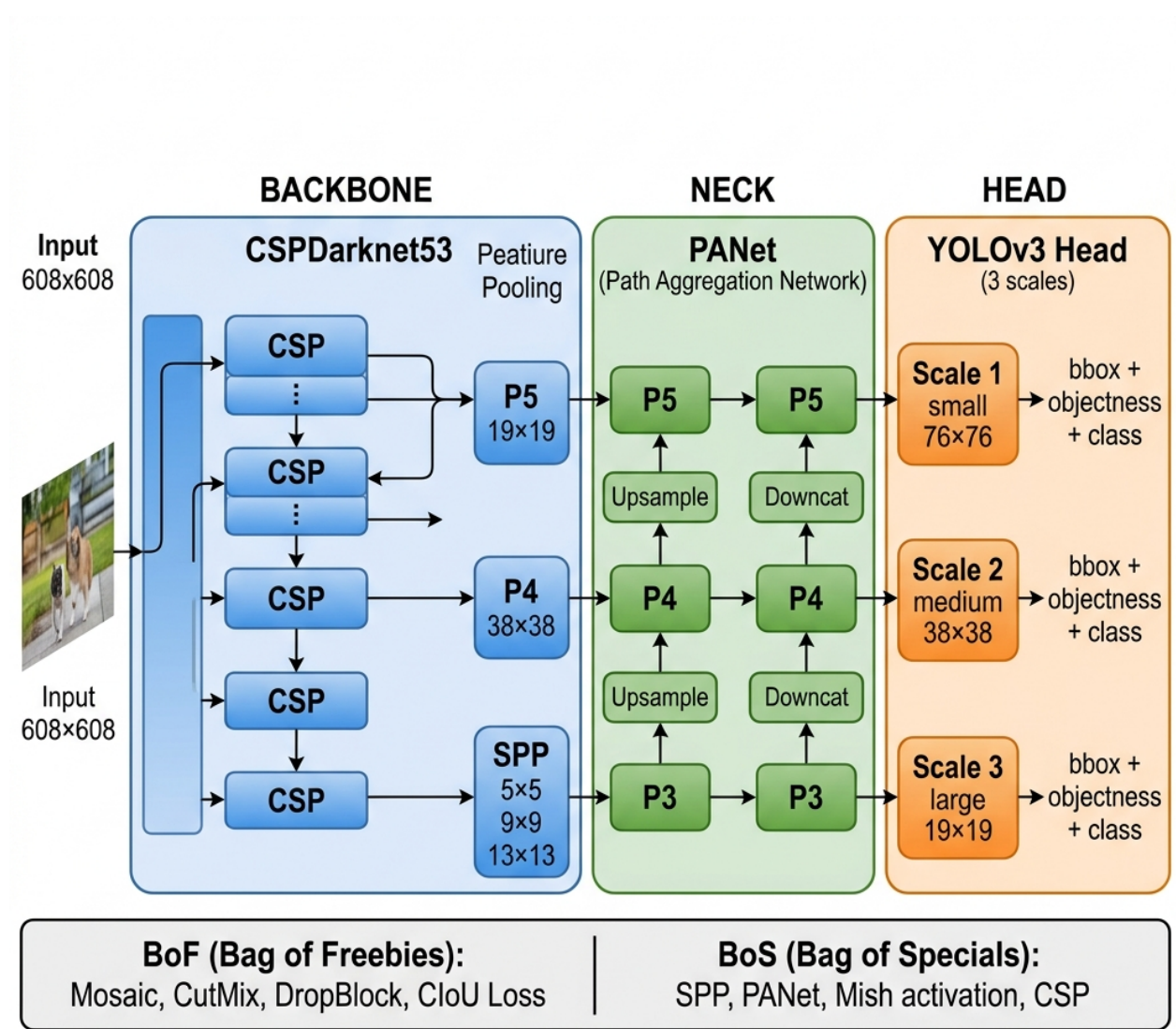
- mAP@50:95 chỉ đạt 33.0% $\rightarrow$ khi yêu cầu IoU cao, khả năng localization vẫn kém.
- Vẫn phụ thuộc NMS để lọc kết quả trùng lặp.

Ví dụ: YOLOv2 giống camera chỉ có 1 ống kính zoom cố định — chụp phong cảnh đẹp nhưng côn trùng nhỏ thì mờ. YOLOv3 giống camera có **3 ống kính** (wide, normal, tele): ống wide bắt cả sân, ống tele bắt rõ con ong trên cành hoa.

3.4 YOLOv4 (2020) — Tập đại thành

Paper: “YOLOv4: Optimal Speed and Accuracy of Object Detection” [4]

Tác giả: Alexey Bochkovskiy, Chien-Yao Wang, Hong-Yuan Mark Liao



Hình 3.2: Kiến trúc YOLOv4: CSPDarknet53 (Backbone) + SPP + PANet (Neck) + YOLOv3 Head [4].

3.4.1 Ý tưởng cốt lõi

Hệ thống hóa hàng trăm kỹ thuật tăng cường thành 2 nhóm: **Bag of Freebies** (miễn phí khi inference) và **Bag of Specials** (tốn thêm chút inference), rồi tìm tổ hợp tối ưu trên single GPU.

3.4.2 Kiến trúc

CSPDarknet53 chia features thành 2 phần xử lý song song rồi gộp lại → giảm **20% computation**. Neck: **SPP** (tăng receptive field) + **PANet** (cải thiện localization).

3.4.3 Nhận xét

Có gì mới?

- **Mosaic Augmentation:** Ghép 4 ảnh thành 1 → model học nhiều ngữ cảnh hơn, giảm batch size (train trên 1 GPU thay vì 8 GPU).
- **CIoU Loss:** Xét cùng lúc overlap, khoảng cách tâm, và tỉ lệ khung → localization chính xác hơn hẳn.
- Đạt **43.5% mAP** trên COCO ở 65 FPS (V100) — nhảy vọt **+10.5%** so với v3, bước nhảy lớn nhất lịch sử YOLO.
- Là YOLO đầu tiên có thể **train trên single GPU** mà vẫn đạt SOTA.

Hạn chế:

- Vẫn viết bằng Darknet (C) → khó tích hợp vào pipeline Python/PyTorch hiện đại.
- Kiến trúc phức tạp, nhiều “kỹ xảo” → khó tùy biến cho người mới.

Ví dụ: Nếu v3 giống chiếc xe đua tốt nhưng chỉ chạy trên đường thẳng, thì v4 giống xe được “độ” toàn diện — thay lốp mới (CIoU), gắn turbo (Mosaic), nâng cấp hộp số (CSPNet) — để chạy nhanh hơn trên **mọi địa hình**, kể cả đường xấu (dataset nhỏ, 1 GPU).

CHƯƠNG 4: THẾ HỆ GIỮA: YOLOv5 – YOLOv8 (2020–2023)

4.1 YOLOv5 (2020) — Dân chủ hóa YOLO

Tác giả: Glenn Jocher / Ultralytics

Framework: PyTorch

Không có paper

4.1.1 Ý tưởng cốt lõi

Chuyển YOLO từ framework Darknet (C) sang **PyTorch**, biến YOLO từ công cụ hàn lâm thành sản phẩm công nghiệp mà ai cũng dùng được chỉ với 3 dòng Python:

```
from ultralytics import YOLO
model = YOLO('yolov5s.pt')
results = model('anh.jpg')
```

4.1.2 Kiến trúc

Backbone: **CSPDarknet53 (modified)** với **C3 module**. Neck: PANet + **SPPF** (3 Max-Pool nối tiếp thay vì song song, nhanh gấp 2×). Head: Anchor-based, 3 scales, có **AutoAnchor** tự tìm anchor tối ưu. Ra mắt **5 kích cỡ model**: Nano, Small, Medium, Large, XLarge.

4.1.3 Nhận xét

Có gì mới?

- **Dễ dùng bậc nhất**: `pip install ultralytics`, export 10+ định dạng (ONNX, TensorRT, CoreML...) → trở thành YOLO phổ biến nhất mọi thời đại.
- **5 sizes** phủ từ IoT (Nano: 1.9M params) đến server (XLarge: 86.7M params).
- **SPPF** tăng tốc SPP gấp 2× mà kết quả tương đương.
- mAP cao nhất đạt **50.7%** (v5x) trên COCO.

Hạn chế:

- Không có paper → không được peer-reviewed, cộng đồng học thuật hoài nghi.
- Kiến trúc gần giống v4, chưa có đột phá kỹ thuật thực sự.

- Vẫn dùng **Anchor-based** — cần tune anchor cho từng dataset.

Ví dụ: YOLOv4 giống siêu xe cần thợ chuyên nghiệp để lái (compile C, config file phức tạp). YOLOv5 giống xe tự lái Tesla — ai cũng lên xe bấm nút là chạy, không cần biết cơ khí.

4.2 YOLOv6 (2022) — Industrial Deployment

Paper: “YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications”

Tác giả: Meituan Vision AI (dịch vụ giao đồ ăn lớn nhất Trung Quốc)

4.2.1 Ý tưởng cốt lõi

Dùng kỹ thuật **Re-parameterization**: kiến trúc khác nhau giữa **training** và **inference**. Khi train dùng multi-branch (học phong phú), khi deploy gộp thành 1 Conv duy nhất (chạy nhanh). Tối ưu đặc biệt cho **quantization INT8** trên GPU server.

4.2.2 Kiến trúc

Backbone: **EfficientRep** (RepVGG blocks). Neck: **Rep-PAN**. Head: **Efficient Decoupled Head** (tách classification/regression). Chuyển sang **Anchor-free** (dự đoán 4 khoảng cách từ tâm, kiểu FCOS).

4.2.3 Nhận xét

Có gì mới?

- **Re-parameterization**: Train với 3 branch song song, deploy gộp thành 1 branch → accuracy cao + inference nhanh.
- **Quantization pipeline** (PTQ → QAT): INT8 chỉ mất **0.2% mAP** nhưng tăng **75% FPS**.
- Vượt v5 ở **mọi scale**, đặc biệt Nano đạt **1187 FPS** (v5n: 602 FPS).

Hạn chế:

- Ecosystem nhỏ hơn v5 rất nhiều, ít tài liệu hướng dẫn.
- Chủ yếu tối ưu cho GPU server (T4), chưa phổ biến trên edge devices.

Ví dụ: Hãy hình dung ôn thi: khi ôn (training), bạn đọc 5 cuốn sách, ghi chú, làm bài tập (multi-branch). Khi vào phòng thi (inference), bạn chỉ cần **1 tờ tóm tắt** chứa tinh hoa từ 5 cuốn — đó là Re-parameterization.

4.3 YOLOv7 (2022) — SOTA Accuracy

Paper: “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art” (CVPR 2023)

Tác giả: Chien-Yao Wang, Alexey Bochkovskiy, Hong-Yuan Mark Liao

4.3.1 Ý tưởng cốt lõi

Mở rộng ELAN thành **E-ELAN** (tăng số nhánh song song mà không phá gradient path gốc), kết hợp với hệ thống **Lead Head + Auxiliary Head** — head phụ chỉ dùng khi training để tạo thêm tín hiệu học, bỏ khi inference.

4.3.2 Kiến trúc

Backbone: **E-ELAN** (Extended Efficient Layer Aggregation). Dùng **RepConvN** (bỏ identity branch để tránh xung đột gradient khi có residual). Head: Coarse-to-Fine label assignment với Lead + Auxiliary heads.

4.3.3 Nhận xét

Có gì mới?

- **E-ELAN:** Tăng cardinality (số nhánh) + shuffle + merge → features phong phú hơn (+0.7% mAP).
- **Auxiliary Head:** Head phụ gán nhãn lỏng hơn → tạo nhiều training signal, **zero inference cost**.
- YOLOv7 (36.9M params) đạt **51.4% mAP** ở 161 FPS — ít params nhất và nhanh nhất so với v5-L, YOLOX-L, PPYOLOE-L.
- YOLOv7-E6E đạt **56.8% mAP** — vượt cả Transformer-based (SWIN-L: 53.9%).

Hạn chế:

- Kiến trúc phức tạp (E-ELAN, RepConvN, dual head) → khó implement và debug.
- Ecosystem không mạnh bằng Ultralytics (v5/v8), ít người dùng ngoài nghiên cứu.

Ví dụ: Auxiliary Head giống **trợ giảng** trong lớp học: giáo viên chính (Lead Head) chấm điểm nghiêm khắc, trợ giảng cho điểm nhẹ nhàng hơn → sinh viên nhận nhiều feedback → học tốt hơn. Khi thi thật (inference), chỉ có giáo viên chính chấm.

4.4 YOLOv8 (2023) — Đa năng nhất

Tác giả: Ultralytics

Không có paper

4.4.1 Ý tưởng cốt lõi

Bỏ hoàn toàn Anchor Boxes (chuyển sang **Anchor-Free**), tách detection head thành 2 nhánh riêng biệt (**Decoupled Head**), và mở rộng thành **framework đa tác vụ**: Detection, Segmentation, Classification, Pose Estimation, OBB, Tracking — tất cả trong cùng 1 API.

4.4.2 Kiến trúc

Backbone: CSPDarknet với **C2f module** (concat output của TẤT CẢ bottlenecks thay vì chỉ output cuối → gradient flow phong phú hơn). Head: Anchor-free, Decoupled, dùng **DFL (Distribution Focal Loss)** — dự đoán phân phối xác suất trên 16 bins thay vì 1 giá trị duy nhất cho mỗi tọa độ.

4.4.3 Nhận xét

Có gì mới?

- **Anchor-Free**: Dự đoán center point + 4 khoảng cách, không cần tune anchor nữa.
- **DFL**: Mô hình hóa “sự không chắc chắn” khi định vị cạnh box → localization chính xác hơn.
- **6 tác vụ** trong 1 framework: Detection, Segmentation, Classification, Pose, OBB, Tracking.
- v8n đạt **37.3% mAP** (+9.3% so với v5n), v8x đạt **53.9%**.

Hạn chế:

- v8-X (53.9%) **thua** v7-E6E (56.8%) về mAP thuần túy.
- Không có paper chính thức → không rõ chi tiết thí nghiệm.

Ví dụ: Nếu cần **accuracy tối đa** → chọn v7. Nếu cần **đa năng + dễ dùng** → chọn v8. DFL giống khi nói “cạnh trái cách tâm khoảng 25px” — thay vì nói “đúng 25px” (1 số), DFL nói “70% khả năng ở 25px, 20% ở 26px, 10% ở 24px” → phản ánh sự không chắc chắn, chính xác hơn.

CHƯƠNG 5: THỂ HỆ MỚI: YOLOv9 – YOLOv12 (2024–2025)

5.1 YOLOv9 (2024) — Chống mất thông tin

Paper: “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information”

Tác giả: Chien-Yao Wang, I-Hau Yeh, Hong-Yuan Mark Liao

5.1.1 Ý tưởng cốt lõi

Giải quyết vấn đề **Information Bottleneck**: khi data đi qua nhiều layers, thông tin bị mất dần (giống trò “truyền tin” qua 20 người). Giải pháp: thêm một **nhánh phụ có khả năng khôi phục thông tin gốc** (PGI) chạy song song khi training, bỏ khi inference.

5.1.2 Kiến trúc

Backbone: **GELAN** (Generalized ELAN) — cho phép dùng bất kỳ loại block nào bên trong (Conv, Bottleneck, Transformer...). Training: thêm **PGI** (Programmable Gradient Information) với auxiliary reversible branch để giữ full information → gradient đáng tin cậy.

5.1.3 Nhận xét

Có gì mới?

- **PGI**: Nhánh phụ reversible giữ toàn bộ thông tin input → gradient luôn chính xác, đặc biệt hiệu quả cho lightweight models (+2.5% AP).
- **GELAN**: Dây chuyền “universal” — lắp block gì vào cũng chạy, linh hoạt hơn ELAN cũ.
- v9 luôn **ít params hơn** v8 mà accuracy cao hơn ở mọi scale: v9-E đạt **55.6%** (v8-X: 53.9%, +1.7%).

Hạn chế:

- PGI chỉ có tác dụng khi training, nên lợi ích phụ thuộc vào chất lượng auxiliary branch.

- Ecosystem nhỏ, chưa được tích hợp rộng rãi vào Ultralytics.

Ví dụ: PGI giống hệ thống **SMS đối chiếu**: main branch là “truyền miệng” qua 20 người (mất thông tin dần), PGI thêm 1 kênh SMS song song — mỗi người đều nhận SMS gốc để đối chiếu và sửa sai kịp thời.

5.2 YOLOv10 (2024) — NMS-Free

Paper: “YOLOv10: Real-Time End-to-End Object Detection”

Tác giả: Ao Wang et al. (Tsinghua University)

5.2.1 Ý tưởng cốt lõi

Loại bỏ hoàn toàn **NMS** (Non-Maximum Suppression) bằng hệ thống **Consistent Dual Assignments**: dùng 2 head — one-to-many (train tốt) và one-to-one (deploy không cần NMS). Khi inference chỉ dùng head one-to-one.

5.2.2 Kiến trúc

Kế thừa v8 nhưng thêm 5 kỹ thuật efficiency: lightweight cls head (depthwise separable), decoupled downsample, rank-guided block, large-kernel conv (7×7), và PSA (partial self-attention, chỉ 50% channels qua attention).

5.2.3 Nhận xét

Có gì mới?

- **NMS-Free**: Không cần bước hậu xử lý → latency ổn định, không phụ thuộc số vật trong ảnh.
- v10-S đạt **46.3% mAP** ở latency 2.49ms — nhanh gấp **2×** so với v8-S+NMS (4.70ms).
- v10-X chỉ cần **29.5M params** (v8-X: 68.2M) — ít hơn **2.3×**.

Hạn chế:

- v10-X (54.4%) **thua** v9-E (55.6%) về mAP — đổi accuracy lấy efficiency.
- Dual head làm tăng complexity khi training.

Ví dụ: NMS giống kiểm tra chất lượng cuối dây chuyền sản xuất — phải dừng lại soi từng sản phẩm, loại phế phẩm. v10 thiết kế dây chuyền “zero defect” — sản phẩm ra là đạt chuẩn ngay, không cần kiểm tra cuối.

5.3 YOLO11 (2024) — Nâng cấp toàn diện

Tác giả: Ultralytics Không có paper Lưu ý: bỏ “v” — YOLO11 thay vì YOLOv11

5.3.1 Ý tưởng cốt lõi

Nâng cấp C2f (v8) thành **C3K2** (dùng 2 kernel sizes: 3×3 và 5×5 trong cùng 1 block) và thêm **C2PSA** (spatial attention) để model tự học “vùng nào quan trọng” trong ảnh.

5.3.2 Kiến trúc

Backbone: CSPDarknet với **C3K2** (multi-scale feature extraction trong cùng 1 block). Neck: thêm **C2PSA** (Cross Stage Partial Spatial Attention) — attention map highlight vùng có vật thể, suppress background.

5.3.3 Nhận xét

Có gì mới?

- **C3K2**: 2 kernel ($3 \times 3 + 5 \times 5$) vừa thấy chi tiết nhỏ vừa thấy bối cảnh rộng — phong phú hơn C2f (chỉ 3×3).
- **C2PSA**: Tập trung vào vùng quan trọng như mắt người, lướt nhanh qua background.
- Cải thiện ở **mọi scale**: 11n đạt **39.5%** (+2.2% so với v8n), 11x đạt **54.7%** (+0.8%).

Hạn chế:

- Cải tiến incremental, chưa có đột phá kiến trúc lớn.
- Không có paper chính thức.

Ví dụ: C2f (v8) giống nhìn qua 1 ống kính cố định — mọi pixel xử lý ngang nhau. C3K2 giống nhìn qua **2 ống kính zoom khác nhau** cùng lúc, còn C2PSA giống **mắt người** — tự động tập trung vào vật thể, lướt nhanh qua nền.

5.4 YOLOv12 (2025) — Kỷ nguyên Attention

Paper: “YOLOv12: Attention-Centric Real-Time Object Detectors”

Tác giả: Yunjie Tian, Qixiang Ye, David Doermann

5.4.1 Ý tưởng cốt lõi

Lần đầu tiên đưa **Attention mechanism** làm thành phần chính của YOLO (thay vì CNN). Dùng **Area Attention** — chia feature map thành n vùng, tính attention trong từng vùng riêng biệt — để giảm complexity xuống $\frac{1}{n}$ mà chỉ mất 0.1% AP.

5.4.2 Kiến trúc

Backbone: **R-ELAN** (Residual ELAN) — thêm residual shortcut với scale factor 0.01 để ổn định training khi dùng attention. Thay Linear+LayerNorm bằng Conv+BatchNorm để tận dụng GPU tốt hơn. Dùng **FlashAttention** để giảm memory.

5.4.3 Nhận xét

Có gì mới?

- **Area Attention**: Giảm $4\times$ complexity so với global attention, chỉ mất 0.1% AP — cách mạng hóa attention cho real-time.
- **R-ELAN**: Giải quyết vấn đề training bất ổn khi kết hợp ELAN + Attention (từ 39.1% $\rightarrow$ **40.6%** ổn định).
- v12-N đạt **40.6%** (v10-N: 38.5%, 11-N: 39.5%), v12-X đạt **55.2%**.

Hạn chế:

- Cần GPU hỗ trợ **FlashAttention** (GPU hiện đại) — không chạy tốt trên edge/NPU.
- Attention operations chưa tối ưu bằng Conv trên nhiều hardware.

Ví dụ: CNN giống đọc sách **từng dòng** (local, nhanh). Attention giống nhìn **cả trang** cùng lúc (global, hiểu sâu nhưng chậm). Area Attention là giải pháp trung gian: chia trang thành 4 phần, nhìn từng phần — nhanh gấp $4\times$ mà vẫn hiểu gần bằng nhìn cả trang.

CHƯƠNG 6: HIỆN TẠI VÀ TƯƠNG LAI: YOLO26 (2026)

Tác giả: Ultralytics

Không có paper chính thức [14]

Phiên bản mới nhất — “trùm cuối” của dòng Ultralytics. Đặt tên theo năm (2026) thay vì số phiên bản.

6.1 Ý tưởng cốt lõi

Triết lý “**Less is More**” + **Edge-First**: bỏ DFL (Distribution Focal Loss) phức tạp, quay về dự đoán trực tiếp, bù bằng 3 training techniques mới (MuSGD, STAL, ProgLoss). Model nhẹ hơn, nhanh hơn trên CPU/NPU, accuracy vẫn SOTA.

6.2 Kiến trúc

Kế thừa YOLO11 nhưng **bỏ DFL** (giảm $16\times$ bandwidth cho box regression head). Bù bằng optimizer mới (**MuSGD**), label assignment thông minh hơn (**STAL**), và loss scheduling động (**ProgLoss**). Thân thiện INT8, chạy tốt trên NPU giá rẻ (Jetson Nano, RK3588).

6.3 Nhận xét

Có gì mới?

- **Bỏ DFL**: Giảm MAC từ 2.15 MB/frame xuống 0.13 MB/frame — nhanh hơn $16\times$ cho box head, thân thiện NPU.
- **MuSGD**: Optimizer mới dùng Muon momentum, hội tụ nhanh hơn AdamW, ít memory hơn.
- **STAL**: Hạ ngưỡng gán nhãn cho vật nhỏ → detect vật nhỏ tốt hơn (drone, camera giám sát).

- **ProgLoss**: Loss weights thay đổi theo epoch — đầu học nhận diện, sau tinh chỉnh vị trí.
- YOLO26-X đạt **~57.2% mAP** — vượt kỷ lục v7-E6E (56.8%) lần đầu sau 4 năm. CPU nhanh hơn 43% so với YOLO11.

Hạn chế:

- Chưa có paper chính thức, số liệu từ benchmark cộng đồng.
- Bỏ DFL có thể giảm localization ở một số trường hợp biên.

Ví dụ: YOLO26 giống chuyển từ “xe sang full option” (DFL, Attention, FlashAttention — chỉ chạy tốt trên đường cao tốc/GPU xịn) sang “xe bán tải bền bỉ” — bỏ bớt tính năng phức tạp, thay bằng kỹ thuật lái giỏi hơn (MuSGD, STAL, ProgLoss) — chạy tốt trên **mọi địa hình** kể cả đường đất (NPU rẻ tiền).

6.4 MuSGD (Muon-enhanced SGD)

Vấn đề: SGD truyền thống cập nhật: $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$. Hội tụ chậm khi loss landscape phức tạp.

Giải pháp — Muon momentum: Muon (phát triển cho LLM) sử dụng momentum trên nhóm trực giao (orthogonal group). Công thức cập nhật:

$$m_{t+1} = \beta \cdot m_t + (1 - \beta) \cdot \nabla_{\theta} \mathcal{L}_t \quad (6.1)$$

$$\hat{m}_{t+1} = \text{NewtonSchulz}(m_{t+1}) \approx U \Sigma^{-1/2} V^T \quad (\text{xấp xỉ SVD}) \quad (6.2)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_{t+1} \quad (6.3)$$

Trong đó NewtonSchulz là phép lặp Newton-Schulz xấp xỉ phân rã SVD trong 5 bước (thay vì $O(n^3)$ của SVD đầy đủ). Hiệu ứng: gradient được **chuẩn hóa trên manifold trọng số**, tránh kẹt ở local minima [14].

Ấn dụ cho người không chuyên

Leo núi trong sương mù: SGD giống bạn chỉ cảm nhận **độ dốc ngay dưới chân** rồi bước xuống — dễ kẹt ở thung lũng nhỏ. MuSGD giống có thêm **la bàn chiến lược** (Muon momentum) nhìn được xu hướng địa hình rộng hơn — khi gặp thung lũng nhỏ, la bàn chỉ bạn bước qua vì phía sau còn thung lũng sâu hơn.

Hiệu quả: Thay thế AdamW (v8–v12) mà accuracy tương đương hoặc cao hơn, ít memory hơn.

6.5 STAL (Small-Target-Aware Label Assignment)

Vấn đề: TAL [16] dùng alignment metric $t = s^\alpha \cdot u^\beta$ (cls score s , IoU u). Vật nhỏ có IoU tự nhiên thấp $\rightarrow$ ít positive samples.

Giải pháp — Dynamic threshold:

$$\tau(A_{gt}) = \tau_{base} \cdot \left(\frac{A_{gt}}{A_{max}} \right)^\gamma, \quad \gamma \in [0.3, 0.5] \quad (6.4)$$

Trong đó A_{gt} là diện tích ground truth, A_{max} là diện tích GT lớn nhất trong batch, γ kiểm soát mức nổi lũng. Khi A_{gt} nhỏ $\rightarrow$ phân số $\frac{A_{gt}}{A_{max}} < 1 \rightarrow \tau$ giảm $\rightarrow$ hạ thấp ngưỡng, giúp nhiều predictions của vật nhỏ được dán nhãn positive hơn [14].

Ẩn dụ cho người không chuyên

Thầy giáo ưu tiên học sinh yếu: Học sinh giỏi (vật lớn) luôn được khen — đã có motivation. Học sinh yếu (vật nhỏ) ít được chú ý. STAL hạ tiêu chuẩn khen thưởng cho nhóm yếu (nới τ), để các em nhận nhiều feedback tích cực hơn $\rightarrow$ dần cải thiện.

Hiệu quả: Đặc biệt tốt cho **drone, camera giám sát tầm xa**, nơi vật thể thường rất nhỏ.

6.6 ProgLoss (Progressive Loss Balancing)

Vấn đề: YOLO loss: $\mathcal{L} = \lambda_{cls}\mathcal{L}_{cls} + \lambda_{box}\mathcal{L}_{box} + \lambda_{obj}\mathcal{L}_{obj}$. Truyền thống dùng λ cố định.

Giải pháp — Adaptive scheduling:

$$\lambda_{cls}(t) = \lambda_{cls}^{max} - (\lambda_{cls}^{max} - \lambda_{cls}^{min}) \cdot \frac{t}{T} \quad (6.5)$$

$$\lambda_{box}(t) = \lambda_{box}^{min} + (\lambda_{box}^{max} - \lambda_{box}^{min}) \cdot \frac{t}{T} \quad (6.6)$$

Với t = epoch hiện tại, T = tổng epochs. Giai đoạn đầu ($t \ll T$): λ_{cls} cao, λ_{box} thấp $\rightarrow$ model học nhận diện trước. Giai đoạn sau ($t \rightarrow T$): λ_{box} tăng $\rightarrow$ tinh chỉnh vị trí.

Ẩn dụ cho người không chuyên

Học lái xe: Tuần đầu: giáo viên dạy **nhận biết biển báo** (cls). Tuần sau: chuyển sang **căn đường, đỗ xe chính xác** (reg). Nếu dạy cả hai cùng tỷ lệ từ đầu (λ cố định), bạn sẽ quá tải.

Hiệu quả: +0.3–0.5% mAP so với fixed λ , đặc biệt khi bỏ DFL.

6.7 Kết quả

Bảng 6.1: YOLO26 so với YOLO11 và YOLOv12 (COCO val, input 640)

Model	Params	mAP@50:95	CPU (ms)	GPU (ms)	Ghi chú
YOLO11-N	2.6M	39.5%	76	1.6	Baseline
YOLO26-N	~2.5M	~40.5%	43	1.4	+1.0%, CPU nhanh 43%
YOLO11-S	9.4M	47.0%	—	2.5	—
YOLO26-S	~9M	~47.5%	—	2.1	+0.5%
YOLOv12-X	59.1M	55.2%	—	12.1	Cần FlashAttention
YOLO26-X	~60M	~57.2%	—	9.8	SOTA, NPU-friendly

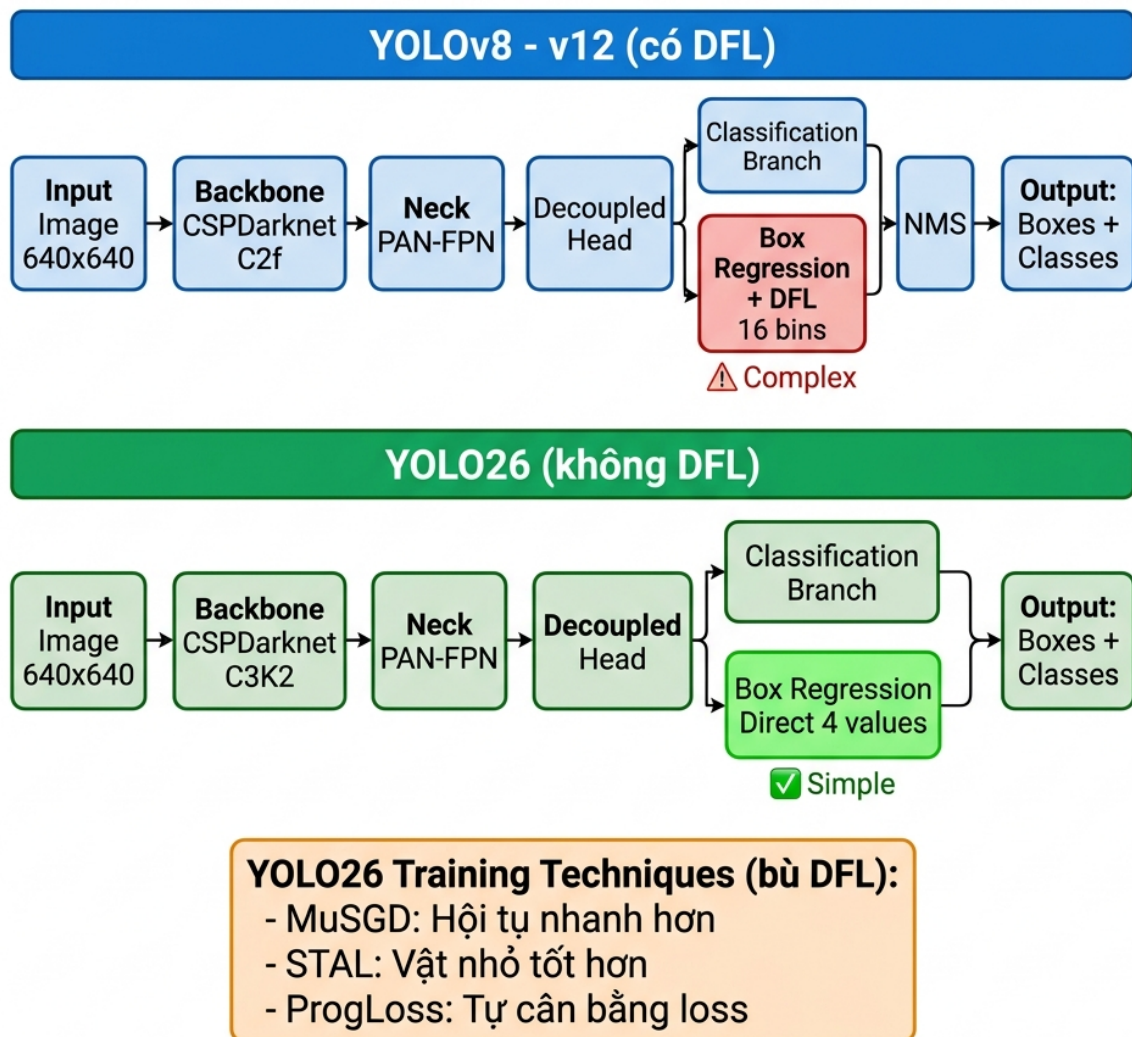
Ghi chú: Số liệu ước tính từ benchmark cộng đồng, chưa có paper chính thức. CPU = ONNX Runtime trên i7-12700. GPU = TensorRT FP16 trên T4 [14].

YOLO26-X (~57.2%) **vượt kỷ lục mAP** của v7-E6E (56.8% [7]) — lần đầu sau 4 năm.

6.8 Tổng kết YOLO26

YOLO26 đại diện cho xu hướng “**less is more**”: bỏ component phức tạp (DFL), thay bằng training techniques (MuSGD, STAL, ProgLoss). Kết quả: model nhẹ hơn, nhanh hơn trên CPU/NPU, accuracy vẫn SOTA — **đúng nghĩa Edge-First Design**.

So sánh Pipeline: YOLOv8-v12 vs YOLO26



Hình 6.1: So sánh pipeline: YOLOv8–v12 (có DFL) vs YOLO26 (không DFL). YOLO26 bù bằng 3 training techniques: MuSGD, STAL, ProgLoss.

CHƯƠNG 7: BƯỚC NGOẶT OPEN-VOCABULARY

7.1 YOLO-World (2024) — Tiên phong

Paper: “YOLO-World: Real-Time Open-Vocabulary Object Detection” [18]

Tác giả: Tencent AI Lab

7.1.1 Ý tưởng cốt lõi

Kết nối YOLO với **CLIP text embeddings** để detect bất kỳ vật nào chỉ bằng mô tả văn bản, không cần train lại. Dùng chiến lược **Prompt-then-Detect**: nạp text prompts 1 lần (offline), sau đó detect real-time.

7.1.2 Kiến trúc

RepVL-PAN (Re-parameterizable Vision-Language PAN) inject text features vào feature pyramid. Tách “hiểu ngôn ngữ” và “detect” thành 2 giai đoạn → inference nhanh.

7.1.3 Nhận xét

Có gì mới?

- Lần đầu tiên YOLO có khả năng **open-vocabulary**: detect vật chưa từng train, chỉ cần gõ tên.
- YOLO-World-L đạt **35.4% AP** (LVIS zero-shot) ở **52 FPS** (V100).
- Mở đường cho YOLOE và kỷ nguyên open-vocabulary detection.

Hạn chế:

- Chỉ hỗ trợ **text prompt**, chưa hỗ trợ visual prompt (đưa ảnh mẫu).
- AP trên LVIS thấp hơn nhiều so với Grounding DINO (chậm hơn 10×).

Ví dụ: YOLO truyền thống giống nhân viên bảo vệ chỉ nhận **80 khuôn mặt đã học**. YOLO-World giống bảo vệ có thể nhận diện bất kỳ ai — chỉ cần bạn **mô tả bằng lời**: “người mặc áo đỏ đội nón xanh”.

7.2 YOLOE (2025) — Real-Time Seeing Anything

Paper: “YOLOE: Real-Time Seeing Anything” (ICCV 2025) [13]

7.2.1 Ý tưởng cốt lõi

Kết hợp **open-vocabulary** + **real-time** với 3 chế độ prompt: text (gõ tên), visual (đưa ảnh mẫu), và prompt-free (tự detect 1203 categories). Giải quyết hạn chế “chỉ text prompt” của YOLO-World.

7.2.2 Kiến trúc

3 module cho 3 chế độ: **RepRTA** (text prompt qua CLIP), **SAVPE** (visual prompt qua semantic + activation features), **LRPC** (prompt-free, pre-compute 1203 categories offline).

7.2.3 Nhận xét

Có gì mới?

- **3 chế độ prompt:** Text (“tìm con chó”), Visual (đưa ảnh con chó mẫu), Prompt-Free (detect mọi thứ tự động).
- YOLOE-L đạt **27.1% AP** (LVIS zero-shot) ở 48 FPS — ngang Grounding DINO (27.4%) nhưng **nhANH gấp 10×**.

Hạn chế:

- AP vẫn thấp hơn nhiều so với COCO closed-set (thang đo khác hoàn toàn).
- Visual prompt cần ảnh mẫu chất lượng tốt để hoạt động hiệu quả.

Ví dụ: Nếu YOLO-World giống bảo vệ nhận diện qua **lời mô tả**, thì YOLOE giống bảo vệ cao cấp hơn — vừa nhận mô tả bằng lời, vừa nhận **ảnh mẫu**, hoặc thậm chí **tự phát hiện** mà không cần ai chỉ dẫn gì.

CHƯƠNG 8: CÁC BIẾN THỂ YOLO

8.1 YOLOX (2021) — Anchor-Free tiên phong

Tác giả: Megvii

Giải thưởng: 1st Place CVPR 2021 Streaming Perception

8.1.1 Ý tưởng cốt lõi

Chứng minh YOLO hoạt động tốt **không cần anchor boxes**, đồng thời giới thiệu **Decoupled Head** (tách cls/reg) và **SimOTA** (dynamic label assignment).

8.1.2 Kiến trúc

Backbone: Modified CSPDarknet. Head: **Decoupled** (2 branches riêng biệt cho classification và regression). Label assignment: **SimOTA** (simplified Optimal Transport Assignment).

8.1.3 Nhận xét

Có gì mới?

- **3 kỹ thuật tiên phong** mà các đời YOLO sau đều adopt: Anchor-Free (v6, v8+), Decoupled Head (v6, v8, v11, v12), SimOTA → TAL (v8, v9, v10+).
- YOLOX-L đạt **50.0% mAP** ở 69 FPS (V100). Nano variant chỉ **0.91M params**.

Hạn chế:

- Không thuộc dòng chính YOLO (Redmon/Wang/Ultralytics) nên ecosystem nhỏ.
- SimOTA tốn computation hơn static assignment.

Ví dụ: YOLOX giống “nhà phát minh” — tự mình không nổi tiếng bằng, nhưng 3 phát minh (Anchor-Free, Decoupled Head, SimOTA) đã được **toàn bộ thế hệ YOLO** sau áp dụng.

8.2 PP-YOLOE (2022) — Baidu Industrial

Tác giả: Baidu

Framework: PaddlePaddle

8.2.1 Ý tưởng cốt lõi

Giới thiệu **TAL (Task Alignment Learning)** — alignment metric $t = s^\alpha \times u^\beta$ đảm bảo classification score cao $\leftrightarrow$ localization tốt. Tối ưu cho triển khai công nghiệp.

8.2.2 Kiến trúc

Backbone: CSPRepResNet. Neck: Custom PAN. Head: Efficient Task-Aligned Head với **TAL label assignment**. Anchor-free.

8.2.3 Nhận xét

Có gì mới?

- **TAL**: Kỹ thuật gán nhãn thông minh nhất thời điểm đó — được **YOLOv8, v9, v10+** adopt làm tiêu chuẩn.
- PP-YOLOE+-L đạt **53.3% mAP** ở 78 FPS. Phổ biến trong công nghiệp Trung Quốc.

Hạn chế:

- Chạy trên PaddlePaddle (Baidu) — ecosystem nhỏ hơn PyTorch rất nhiều.
- Ít được cộng đồng quốc tế sử dụng ngoài Trung Quốc.

Ví dụ: TAL giống hệ thống chấm điểm “2 tiêu chí”: vừa phải **nhận đúng người** (cls score cao), vừa phải **khoanh đúng vị trí** (IoU cao). Chỉ khi cả 2 đều tốt mới được coi là positive — tránh tình trạng “nhận đúng nhưng khoanh sai”.

8.3 YOLO-NAS (2023) — AI thiết kế AI

Tác giả: Deci AI (Israel)

8.3.1 Ý tưởng cốt lõi

Dùng **AutoNAC** (Neural Architecture Search) để tự động khám phá $\sim 10^{14}$ kiến trúc và tìm tổ hợp tối ưu. Thiết kế sẵn cho quantization ngay từ đầu.

8.3.2 Kiến trúc

Backbone/Neck/Head đều do **AutoNAC tự thiết kế** — không phải con người. Tối ưu đồng thời cho accuracy, latency, và quantization-friendliness.

8.3.3 Nhận xét

Có gì mới?

- **NAS-designed**: Kiến trúc do AI tìm ra, không phải con người thiết kế thủ công.
- INT8 quantization chỉ mất **0.4–0.6% mAP** (các model khác mất 1–2%).

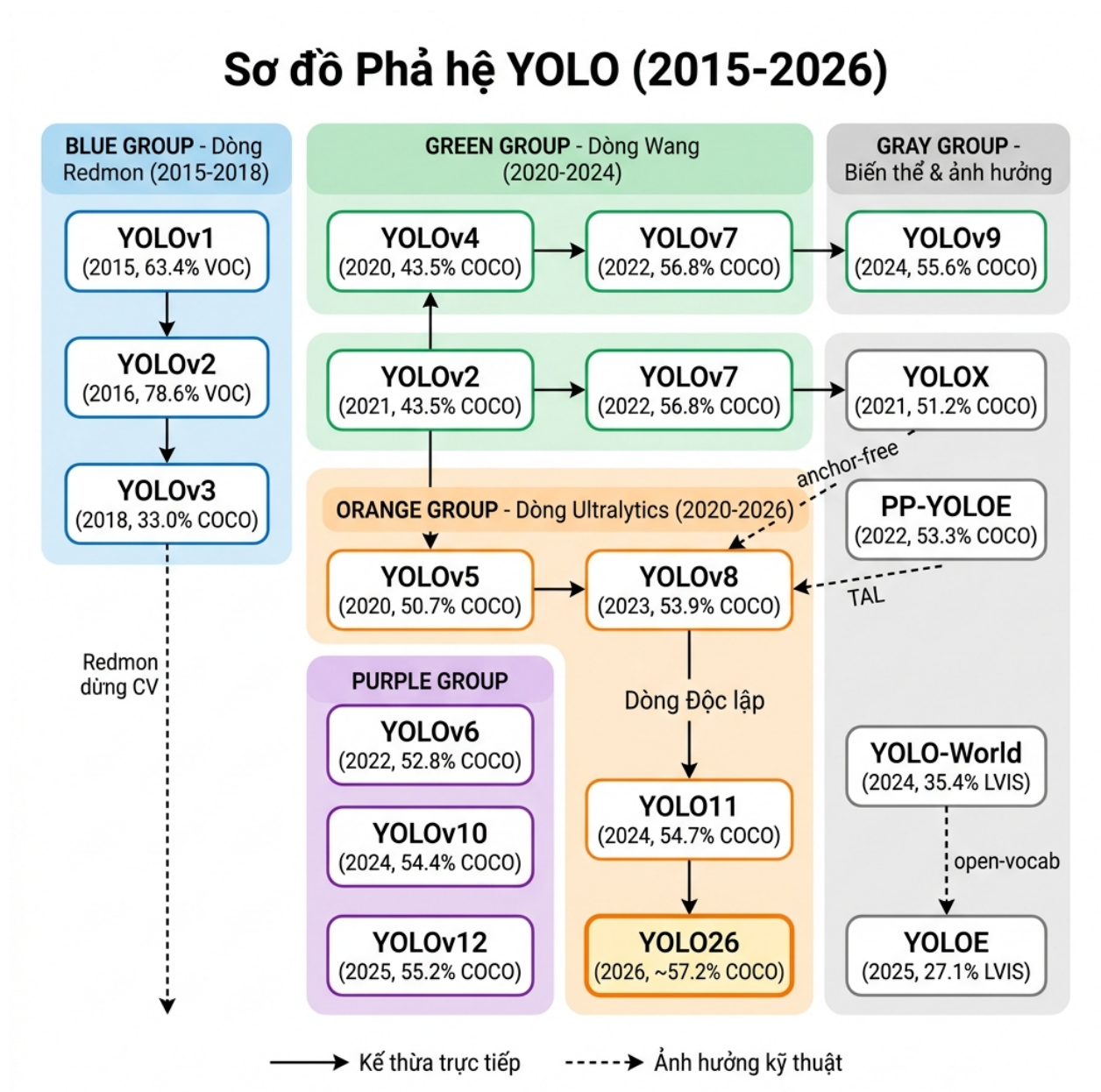
Hạn chế:

- Weights không hoàn toàn open-source, cần license Deci AI cho một số tính năng.
- Kiến trúc NAS khó giải thích và tùy biến.

Ví dụ: Nếu YOLOv4–v12 giống xe do **kỹ sư con người** thiết kế (dựa trên kinh nghiệm và thí nghiệm), thì YOLO-NAS giống xe do **AI tự thiết kế** — thử hàng nghìn tỉ tổ hợp rồi chọn ra xe tối ưu nhất.

CHƯƠNG 9: SƠ ĐỒ PHẢ HỆ YOLO VÀ KẾT LUẬN

9.1 Sơ đồ phả hệ YOLO (2015–2026)



Hình 9.1: Sơ đồ phả hệ các phiên bản YOLO (2015–2026). Đường liền: kế thừa trực tiếp. Đường nét đứt: ảnh hưởng kỹ thuật.

Lưu ý: VOC mAP@50 (v1–v2) và COCO mAP@50:95 (v3+) là hai thang đo hoàn toàn khác nhau, không so sánh trực tiếp. LVIS zero-shot (YOLO-World, YOLOE) cũng là thang đo riêng.

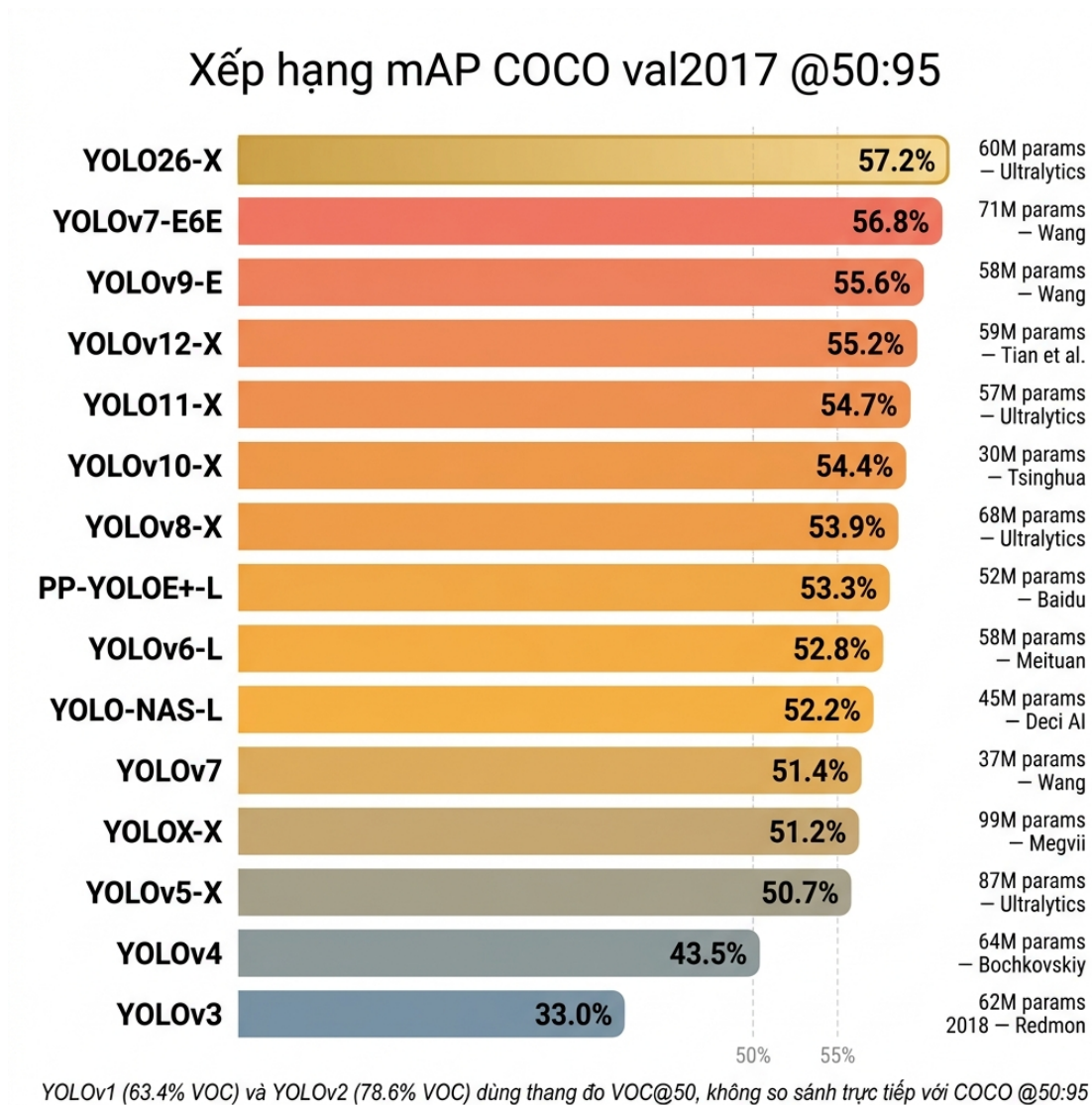
9.2 Bảng tổng quan tất cả phiên bản

Bảng 9.1: Tổng quan các phiên bản YOLO (2015–2026)

Version	Năm	mAP VOC@50	mAP COCO@50:95	FPS (GPU)	Params	Anchor	NMS	Paper
v1 [1]	2015	63.4%	—	45 (Titan X)	~45M	Grid	Cần	Có
v2 [2]	2016	78.6%	—	67 (Titan X)	~50M	Có	Cần	Có
v3 [3]	2018	—	33.0%	20 (Titan X)	~62M	Có	Cần	Có
v4 [4]	2020	—	43.5%	65 (V100)	~64M	Có	Cần	Có
v5 [5]	2020	—	50.7%	98 (V100)	86.7M	Có	Cần	Không
v6 [6]	2022	—	52.5%	116 (T4)	~58M	Không	Cần	Có
v7 [7]	2022	—	56.8%	36 (V100)	~71M	Có	Cần	Có
v8 [8]	2023	—	53.9%	83 (T4)	68.2M	Không	Cần	Không
v9 [9]	2024	—	55.6%	57 (T4)	58.1M	Không	Cần	Có
v10 [10]	2024	—	54.4%	96 (T4)	29.5M	Không	Không	Có
YOLO11 [11]	2024	—	54.7%	79 (T4)	56.9M	Không	Cần	Không
v12 [12]	2025	—	55.2%	51 (T4)	59.1M	Không	Không	Có
YOLO26 [14]	2026	—	~57.2%	~102 (T4)*	~60M	Không	Không	Không

* Ước tính. FPS = model size lớn nhất (v7-E6E, v5x, v8x...) trừ v1–v3 (model duy nhất). GPU ghi trong ngoặc. Lưu ý: Titan X, V100, T4 là 3 thế hệ GPU khác nhau — FPS giữa chúng **không so sánh trực tiếp được**. v1–v2 dùng mAP VOC@50, v3+ dùng mAP COCO@50:95 — hai thang đo hoàn toàn khác nhau.

9.3 Xếp hạng mAP COCO



Hình 9.2: Xếp hạng mAP COCO val2017 @50:95 — tất cả phiên bản YOLO. Thanh trên cùng = model mạnh nhất hiện tại. YOLOv1–v2 dùng thang đo VOC@50 khác biệt.

9.4 Tiến hóa công nghệ qua các phiên bản



Hình 9.3: Tiến hóa công nghệ YOLO (2015–2026): Backbone, Anchor, NMS, Loss, Framework, Optimizer, và Label Assignment thay đổi qua từng phiên bản.

9.5 Kết luận

Qua 11 năm phát triển (2015–2026) và 14+ phiên bản, YOLO đã chứng minh: **không có “model tốt nhất” — chỉ có “model phù hợp nhất”**.

Ba dòng phát triển song song — Redmon (v1–v3), Wang (v4,v7,v9), Ultralytics (v5,v8,v11,v26)

Khảo sát toàn diện sự phát triển của YOLO

— cho thấy sức sống phi thường của triết lý “You Only Look Once”.

Các bước ngoặt quan trọng nhất:

- **YOLOv1** [1]: Chứng minh single-stage detection khả thi
- **YOLOv4** [4]: Hệ thống hóa BoF/BoS — phương pháp luận nghiên cứu mẫu mực
- **YOLOv8** [8]: Anchor-free + 6 tasks — dân chủ hóa object detection
- **YOLOv10** [10]: NMS-free — giải quyết vấn đề tồn tại 9 năm
- **YOLOv12** [12]: Attention-centric — mở kỷ nguyên mới
- **YOLO26** [14]: Edge-First + “less is more” — bỏ DFL, bù bằng training techniques

*“Chênh lệch 1–3% mAP giữa các version trên COCO có thể không có ý nghĩa thống kê trên dataset thực tế. **Data chất lượng quan trọng hơn chọn model.**”*

Tài liệu tham khảo

- [1] J. Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” CVPR 2016. arXiv:1506.02640
- [2] J. Redmon, A. Farhadi, “YOLO9000: Better, Faster, Stronger,” CVPR 2017. arXiv:1612.08242
- [3] J. Redmon, A. Farhadi, “YOLOv3: An Incremental Improvement,” 2018. arXiv:1804.02767
- [4] A. Bochkovskiy et al., “YOLOv4: Optimal Speed and Accuracy of Object Detection,” 2020. arXiv:2004.10934
- [5] G. Jocher, “YOLOv5,” Ultralytics, 2020. github.com/ultralytics/yolov5
- [6] C. Li et al., “YOLOv6: A Single-Stage Object Detection Framework,” 2022. arXiv:2209.02976
- [7] C.-Y. Wang et al., “YOLOv7: Trainable Bag-of-Freebies,” CVPR 2023. arXiv:2207.02696
- [8] G. Jocher, “YOLOv8,” Ultralytics, 2023. github.com/ultralytics/ultralytics
- [9] C.-Y. Wang et al., “YOLOv9: Learning What You Want to Learn Using PGI,” 2024. arXiv:2402.13616
- [10] A. Wang et al., “YOLOv10: Real-Time End-to-End Object Detection,” 2024. arXiv:2405.14458
- [11] G. Jocher, “YOLO11,” Ultralytics, 2024. github.com/ultralytics/ultralytics
- [12] Y. Tian et al., “YOLOv12: Attention-Centric Real-Time Object Detectors,” 2025. arXiv:2502.12524
- [13] “YOLOE: Real-Time Seeing Anything,” ICCV 2025. arXiv:2503.07465
- [14] G. Jocher, “YOLO26,” Ultralytics, 2026. github.com/ultralytics/ultralytics
- [15] Z. Ge et al., “YOLOX: Exceeding YOLO Series in 2021,” 2021. arXiv:2107.08430
- [16] S. Xu et al., “PP-YOLOE: An Evolved Version of YOLO,” 2022. arXiv:2203.16250
- [17] “YOLO-NAS,” Deci AI, 2023. github.com/Deci-AI/super-gradients
- [18] T. Cheng et al., “YOLO-World: Real-Time Open-Vocabulary Object Detection,” 2024. arXiv:2401.17270

PHỤ LỤC: BẢNG THUẬT NGỮ

Dưới đây là các thuật ngữ kỹ thuật thường gặp trong lĩnh vực YOLO và Object Detection, sắp xếp theo nhóm chủ đề.

Kiến trúc mạng (Architecture)

Thuật ngữ	Giải thích
Backbone	Phần “xương sống” của model, trích xuất features từ ảnh đầu vào. Ví dụ: Darknet, CSPDarknet, ResNet, EfficientNet.
Neck	Kết nối Backbone và Head, kết hợp features ở nhiều scales. Ví dụ: FPN, PAN, BiFPN.
Head	Phần cuối cùng, dự đoán bounding boxes và class probabilities. Có 2 loại: coupled (chung) và decoupled (tách riêng cls/reg).
CNN	Convolutional Neural Network — mạng neural dùng phép tích chập (convolution) để trích xuất features từ ảnh. Mỗi layer học nhận diện patterns (cạnh, góc, hình dạng).
Convolution	Phép tính trượt 1 bộ lọc (kernel) qua ảnh, tạo feature map. Kernel 3×3 nhìn vùng 3×3 pixels.
Depthwise Conv	Convolution áp dụng riêng cho từng channel (thay vì tất cả channels cùng lúc) — giảm computation đáng kể.
Residual Connection	Đường tắt (shortcut) nối input trực tiếp đến output: $y = F(x) + x$. Giải quyết vanishing gradient khi mạng rất sâu (ResNet).
CSP	Cross Stage Partial — chia features thành 2 phần: 1 phần qua processing, 1 phần bypass → giảm 20% computation.
FPN	Feature Pyramid Network — kết hợp features từ nhiều scales (top-down pathway) để detect vật ở mọi kích thước.
PAN / PANet	Path Aggregation Network — thêm bottom-up pathway vào FPN → localization signals mạnh hơn.

Thuật ngữ	Giải thích
SPP / SPPF	Spatial Pyramid Pooling — MaxPool nhiều kích thước → tăng receptive field. SPPF: phiên bản nhanh hơn (nối tiếp thay song song).
Re-parameterization	Kỹ thuật training multi-branch, inference single-branch. Gộp nhiều Conv thành 1 Conv duy nhất bằng phép cộng ma trận.
Attention	Cơ chế cho model “tập trung” vào vùng quan trọng. Self-attention so sánh mọi vị trí với nhau (global receptive field).
FlashAttention	Thuật toán tính attention tối ưu memory — giảm memory footprint đáng kể, cho phép dùng attention trên feature maps lớn.

Kỹ thuật Detection

Thuật ngữ	Giải thích
Bounding Box	Hình chữ nhật bao quanh vật thể, xác định vị trí (x, y, width, height).
Anchor Box	Hộp mẫu (template) được định nghĩa trước. Model dự đoán offsets so với anchor thay vì tọa độ tuyệt đối. v2-v7 dùng anchor, v8+ bỏ anchor.
Anchor-free	Không dùng anchor. Dự đoán trực tiếp center point + 4 distances (khoảng cách từ tâm đến 4 cạnh). Đơn giản hơn anchor-based.
NMS	Non-Maximum Suppression — thuật toán loại bỏ bounding boxes trùng lặp. Giữ box confidence cao nhất, bỏ các boxes có IoU lớn.
NMS-free	Không cần NMS — model tự output 1 box/vật. Từ v10 trở đi. Dùng dual assignment (o2m + o2o).
Decoupled Head	Tách classification và regression thành 2 branches riêng biệt. YOLOX giới thiệu, v6, v8+ adopt.
Label Assignment	Chiến lược gán prediction cho ground truth khi training. SimOTA (động), TAL (task-aligned), STAL (small-target-aware).
End-to-End	Pipeline từ input đến output không cần bước xử lý thủ công trung gian (ví dụ: không cần NMS).

Thuật ngữ	Giải thích
Multi-scale	Dự đoán ở nhiều kích thước (scales). YOLOv3+: 3 scales (lớn/trung/nhỏ) để detect vật ở mọi kích thước.
One-stage / Two-stage	One-stage: 1 lần forward pass ra kết quả (YOLO). Two-stage: region proposal + classification (Faster R-CNN).

Metrics (Chỉ số đánh giá)

Thuật ngữ	Giải thích
IoU	Intersection over Union — đo mức trùng lặp giữa predicted box và ground truth. $IoU = \frac{Giao}{Hop}$. Từ 0 (không trùng) đến 1 (trùng hoàn hảo).
mAP	mean Average Precision — trung bình AP trên tất cả classes. Metric chính đánh giá detection.
mAP@50	AP ở ngưỡng $IoU \geq 0.5$. Dùng trên VOC.
mAP@50:95	Trung bình AP ở IoU từ 0.5 đến 0.95 (bước 0.05). Metric chính trên COCO — khắt khe hơn mAP@50.
FPS	Frames Per Second — số ảnh xử lý mỗi giây. ≥ 30 FPS = real-time.
Params	Số lượng tham số của model. Ví dụ: 3.2M = 3.2 triệu tham số. Ít params = model nhẹ.
FLOPs / GFLOPs	Floating Point Operations — số phép tính cần thực hiện. Đo computational cost. 1 GFLOPs = 1 tỷ phép tính.
Latency	Thời gian xử lý 1 ảnh (ms). Khác FPS vì bao gồm cả pre/post-processing.

Training (Huấn luyện)

Thuật ngữ	Giải thích
Batch Normalization	Chuẩn hóa output mỗi layer theo mini-batch $\rightarrow$ training ổn định hơn, nhanh hội tụ hơn. YOLOv2 thêm BN: +2% mAP.
Mosaic	Data augmentation ghép 4 ảnh thành 1 $\rightarrow$ model thấy nhiều context, giảm batch size cần thiết. YOLOv4 giới thiệu.
CutMix	Cắt 1 vùng từ ảnh A dán vào ảnh B, mix labels theo tỷ lệ diện tích.
Augmentation	Biến đổi ảnh khi training (xoay, lật, zoom, đổi màu...) để model tổng quát hơn.
Transfer Learning	Dùng model đã train trên dataset lớn (ImageNet) $\rightarrow$ fine-tune trên dataset nhỏ của mình. Tiết kiệm thời gian, ít data hơn.
Quantization	Giảm precision từ FP32 (32-bit) xuống INT8 (8-bit) $\rightarrow$ model nhẹ hơn 4 $\times$, nhanh hơn, nhưng có thể mất accuracy.
PTQ	Post-Training Quantization — quantize sau khi train xong. Nhanh nhưng mất accuracy.
QAT	Quantization-Aware Training — train lại với simulated quantization $\rightarrow$ giữ accuracy tốt hơn PTQ.
BoF / BoS	Bag of Freebies (tăng training cost, không tăng inference) / Bag of Specials (tăng nhẹ inference cost). YOLOv4 hệ thống hóa.

Loss Functions (Hàm mất mát)

Thuật ngữ	Giải thích
MSE / SSE	Mean/Sum Squared Error — đo sai số bình phương. v1–v3 dùng cho box coordinates.
IoU Loss	$\mathcal{L} = 1 - IoU$. Trực tiếp tối ưu IoU thay vì tọa độ riêng lẻ.
GIoU	Generalized IoU — xét cả vùng bao ngoài 2 boxes. Giải quyết trường hợp 2 box không giao nhau ($IoU = 0$ nhưng $GIoU \neq 0$).
CIoU	Complete IoU — thêm khoảng cách tâm + aspect ratio vào GIoU. $\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2}{c^2} + \alpha v$. YOLOv4+ dùng.

Thuật ngữ	Giải thích
DFL	Distribution Focal Loss — dự đoán phân phối xác suất cho box coordinates (16 bins) thay vì 1 giá trị. v8–v12 dùng, YOLO26 bỏ.
BCE	Binary Cross-Entropy — loss cho classification. v3+ dùng BCE thay Softmax để hỗ trợ multi-label.

Deployment (Triển khai)

Thuật ngữ	Giải thích
ONNX	Open Neural Network Exchange — format chuẩn để export model, chạy được trên nhiều frameworks/hardware.
TensorRT	NVIDIA SDK tối ưu inference trên GPU NVIDIA. Tăng 2–5× speed so với PyTorch.
Edge deployment	Triển khai model trên thiết bị nhỏ (mobile, camera, drone) thay vì server lớn. Cần model nhẹ, ít computation.
FP16 / FP32	Float 16-bit / 32-bit precision. FP16 nhanh hơn, ít memory hơn, gần như không mất accuracy.
INT8	Integer 8-bit. Nhẹ hơn FP32 gấp 4× nhưng có thể mất 0.5–2% accuracy.